🗺️ RTAB-Map Vision and LiDAR Fusion 3D Mapping (RTAB-Map SLAM)

1. Course Content

Combining the RGB-D image from the **Angstrong Nuwa-HP60C 3D Depth Camera** and **LiDAR 2D scan point cloud**, run the RTAB-Map (Real-Time Appearance-Based Mapping) algorithm to construct a composite spatial map database (`rtabmap.db`) containing high-density 3D point clouds and 2D grids.

> **Important**
>
> This course is only applicable to users with the **Angstrong Nuwa-HP60C Depth Camera** version.

---

2. RTAB-Map Algorithm Core Principles

RTAB-Map is a real-time graph-optimized SLAM system based on appearance loop closure detection, with three core mechanisms:

1. **Multi-modal sensor frontend fusion**: Synchronously collect LiDAR Scan, Depth, and RGB, extract visual feature points (ORB/SURF/SIFT) and combine with wheel/EKF odometry to estimate pose;
2. **Bayesian loop closure detection and global pose graph optimization**: When the robot returns to a historical area, use visual Bag-of-Words (BoW) to quickly retrieve and match keyframes and trigger graph optimization to eliminate cumulative drift.

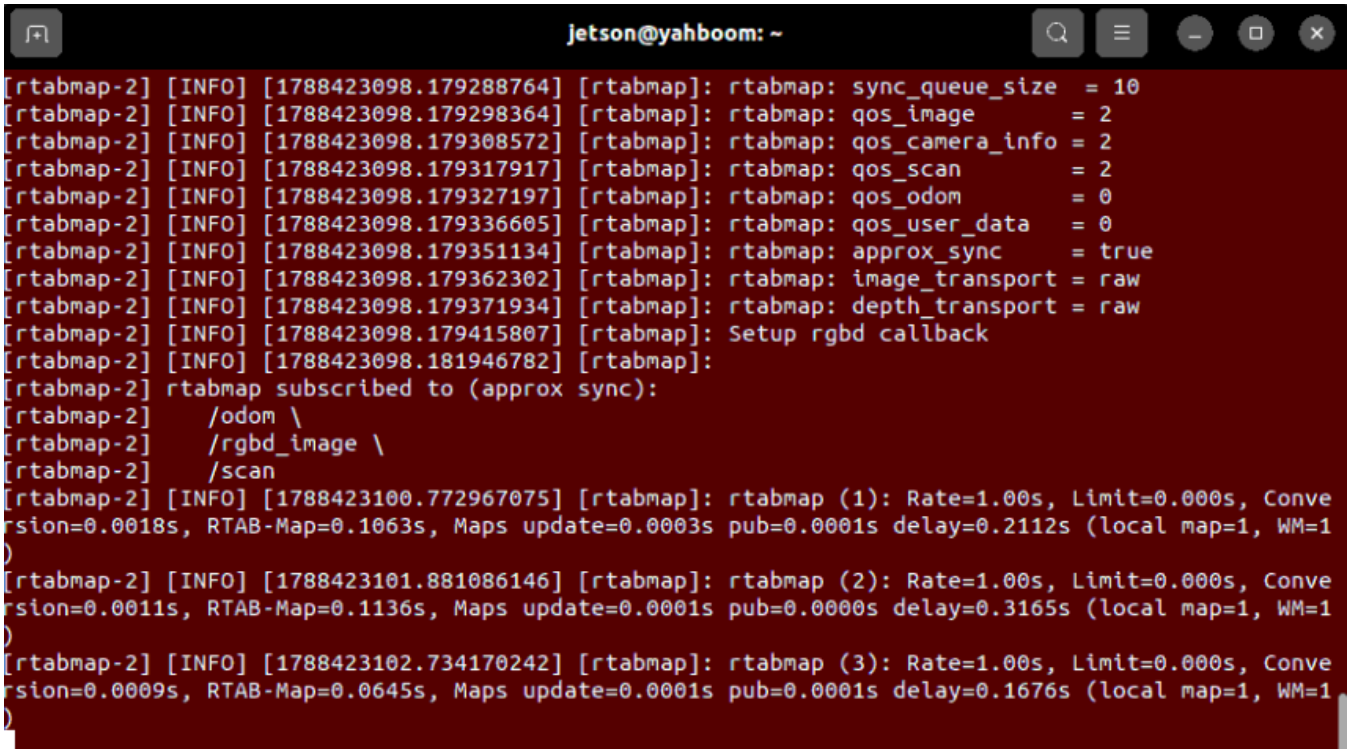
---

3. Preparation and Running

3.1 Start RTAB Mapping Core

Run in **Terminal 1** (start Nuwa depth camera, chassis driver, EKF filtering and LiDAR, mapping):

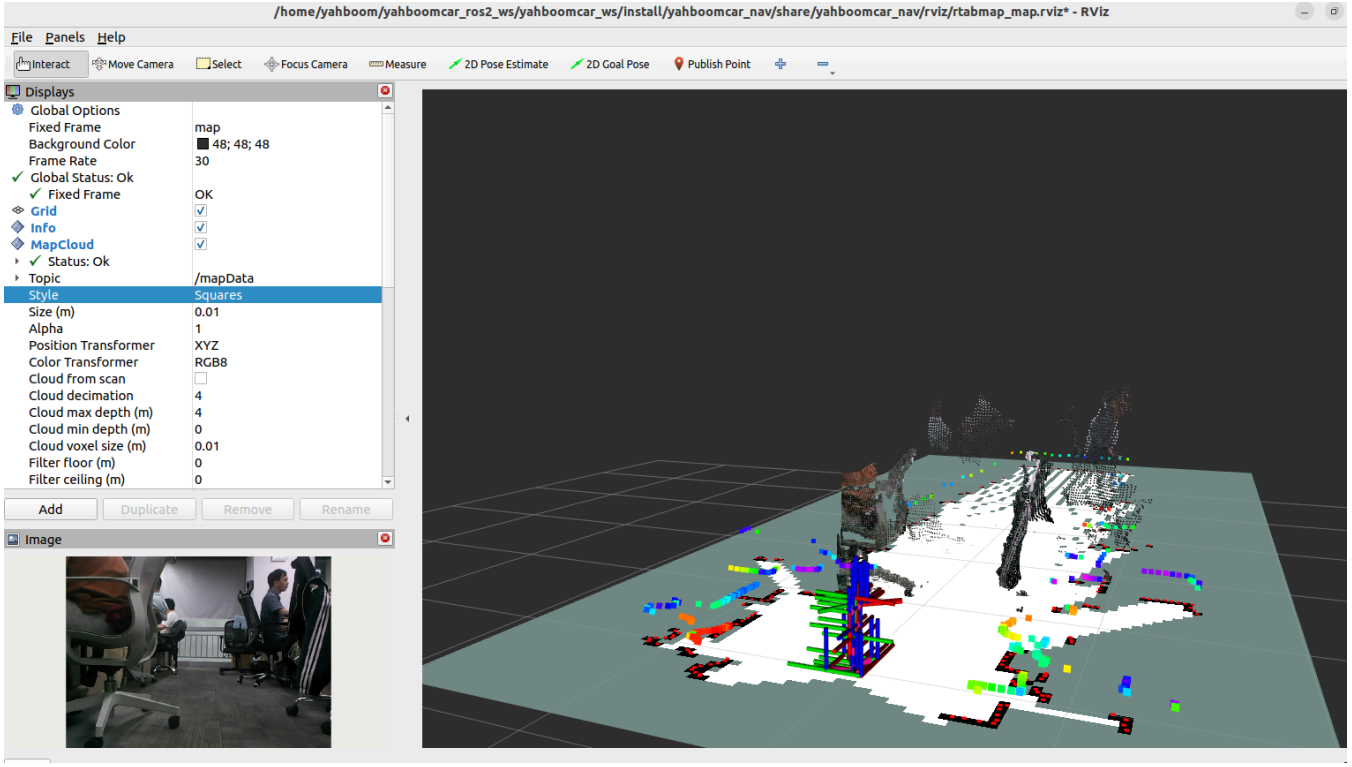
```
ros2 launch yahboomcar_nav map_rtabmap_launch.py
```



3.2 Start RViz 3D Mapping Visualization

Run in **Terminal 2**: **(Orin motherboard does not need to run this command, RDK and Raspberry Pi motherboards need to run in virtual machine)**

```
ros2 launch yahboomcar_nav display_rtabmap_map_launch.py
```



3.3 Start Keyboard Control for Mapping

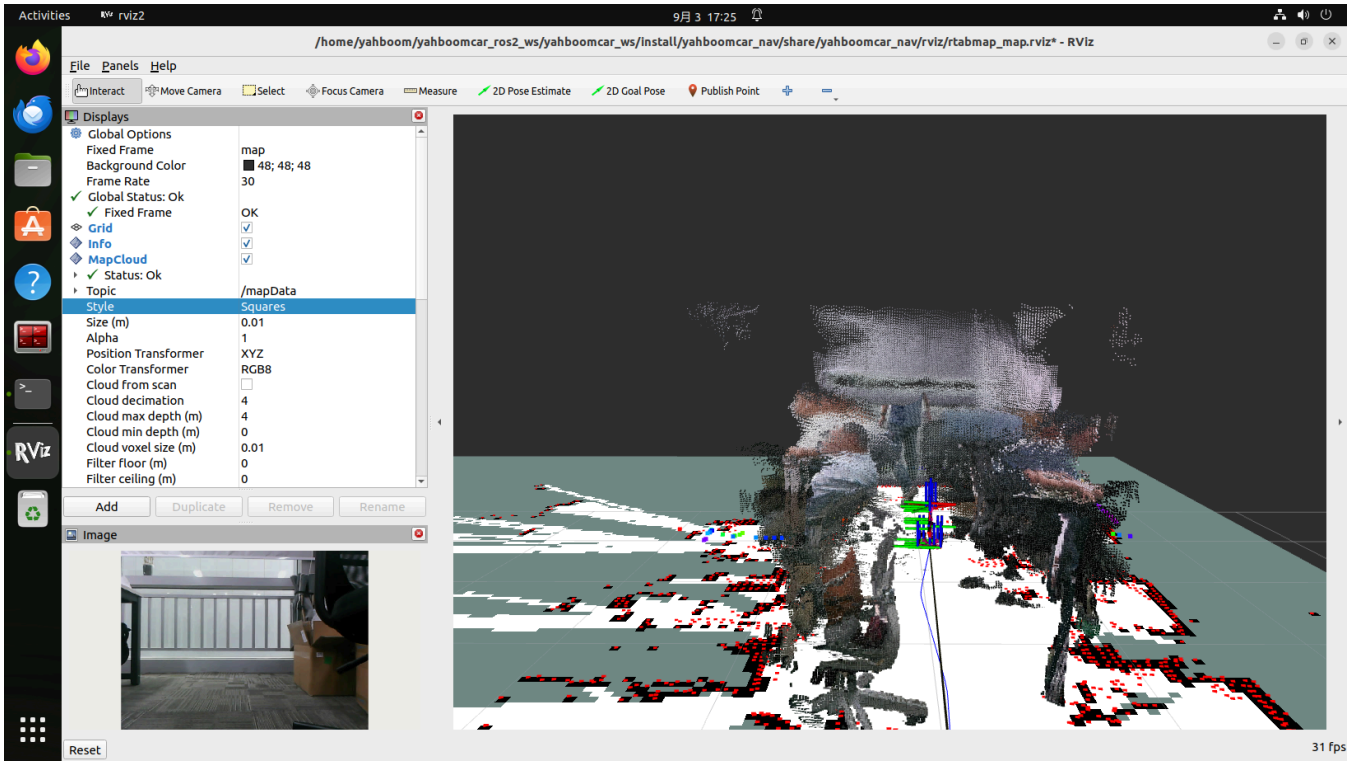
Run in **Terminal 3**:

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

---

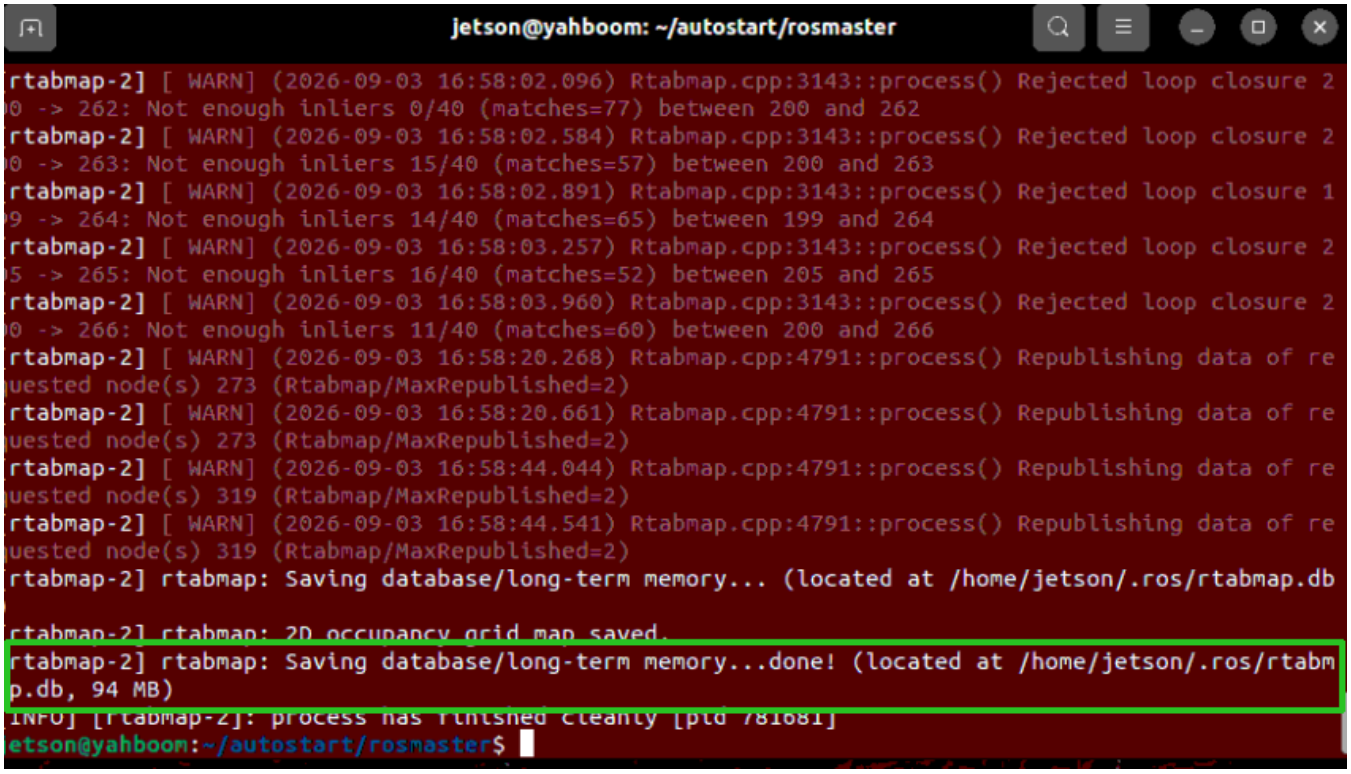
4. Hands-on Demo and Mapping Operation

1. Keep the keyboard control terminal focused, press `i` (forward), `u` (forward-left), `o` (forward-right), `,` (backward) to control the robot to slowly cruise in the environment;
2. Observe the gradually constructed 3D color point cloud model and 2D grid boundary in the RViz interface;



3. **Loop closure construction**: Control the robot to make a round trip back to the starting point, and observe the system automatically align and merge point clouds after detecting the loop closure;
4. **Save map database**: After mapping is complete, press `Ctrl+C` directly in each terminal window to stop the program, and RTAB-Map will automatically save the complete graph-optimized pose and point cloud database to:

   ```
   ~/map/rtabmap.db  (or ~/.ros/rtabmap.db)
   ```

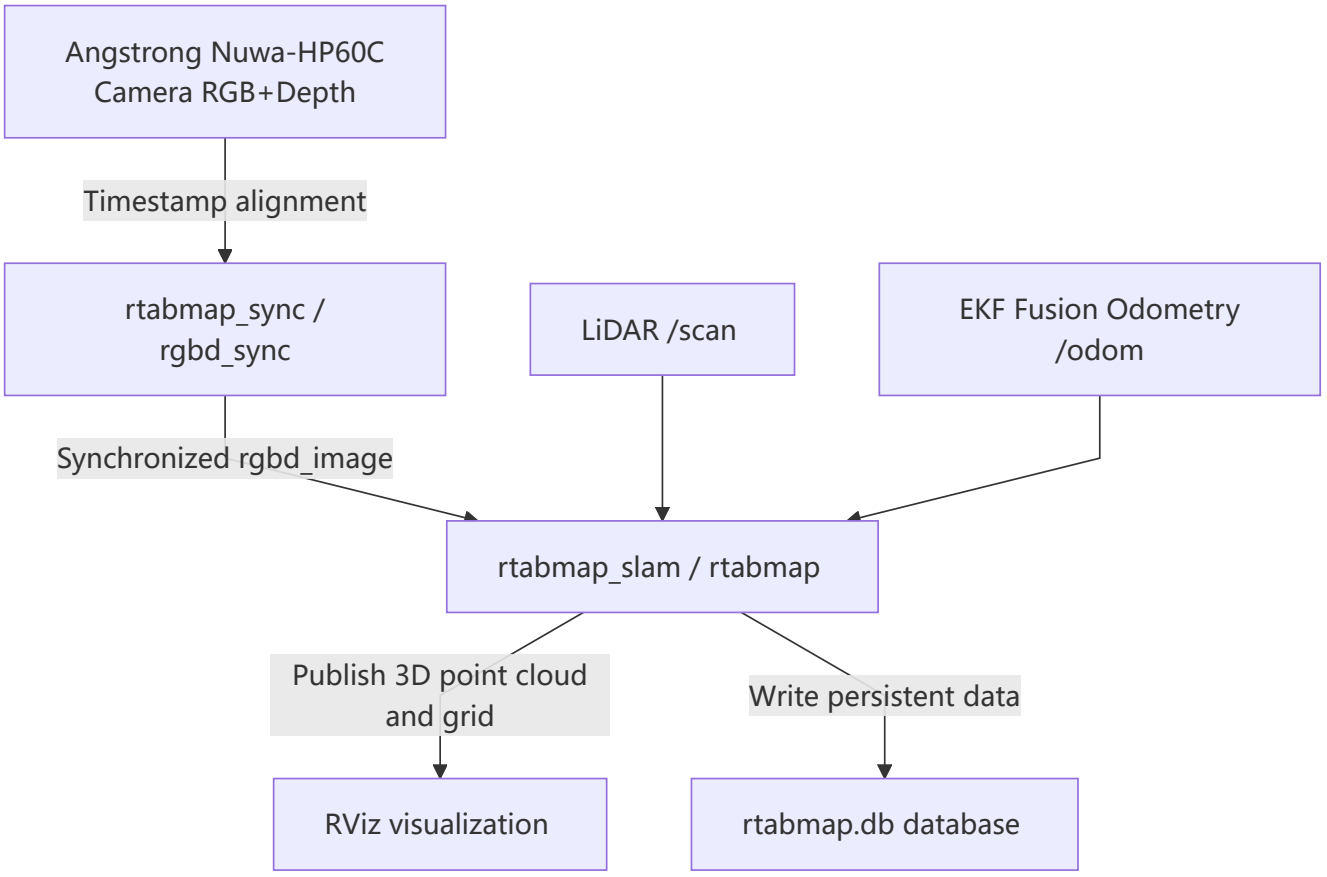


5. Core Source Code Analysis and Architecture Design

Source Code Reference:

- `yahboomcar_ws/src/yahboomcar_nav/launch/rtab/map_rtabmap_launch.py`
- `yahboomcar_ws/src/yahboomcar_nav/launch/rtab/rtabmap_sync_launch.py`

The entire mapping system adopts a loosely-coupled architecture design: the underlying sensors are centrally scheduled in `car_base.launch.py`, and the mapping top-level `map_rtabmap_launch.py` focuses on RTAB-Map synchronization and graph optimization:



5.1 RGB-D Multi-stream Soft Timestamp Synchronization (`rgbd_sync`)

Because the HP60C camera's `camera_info` and RGB/Depth have slight timestamp jitter, the system performs loose approximate timestamp synchronization through the `rgbd_sync` node in `rtabmap_sync`:

```
# rtabmap_sync_launch.py -- Sensor input synchronization node
Node(
    package='rtabmap_sync', executable='rgbd_sync', output='screen',
    parameters=[{
        'approx_sync': True,             # Enable soft timestamp approximate alignment
        'approx_sync_max_interval': 0.0, # Keep unbounded approximate sync, tolerate camera
frame rate jitter
        'topic_queue_size': 20,
        'sync_queue_size': 20,
    }],
    remappings=[
        ('rgb/image', '/ascamera_hp60c/camera_publisher/rgb0/image'),
        ('rgb/camera_info', '/ascamera_hp60c/camera_publisher/rgb0/camera_info'),
        ('depth/image', '/ascamera_hp60c/camera_publisher/depth0/image_raw'),
        ('odom', '/odom')
    ]
)
```

5.2 Constrained Ground 3DoF Optimized SLAM Core Parameters

For ground Ackermann/differential mobile robots, the parameter configuration forcibly locks 3 degrees of freedom (X, Y, Yaw) to eliminate false drift from pitch and roll caused by ground bumps:

```
# rtabmap_sync_launch.py —— SLAM parameter configuration snippet
parameters = {
    'frame_id': 'base_footprint',
    'map_frame_id': 'map',
    'subscribe_rgbd': True,
    'subscribe_scan': True,             # Fuse LiDAR 2D contour
    'approx_sync': True,

    # 【3DoF constraint】Force constraint on 2D ground plane movement optimization
    'Reg/Strategy': '1',                # 1=ICP-based LiDAR auxiliary registration
    'Reg/Force3DoF': 'true',            # Ignore Roll/Pitch
    'RGBD/ForceOdom3DoF': 'true',

    # 【Turn and keyframe trigger mechanism】
    'Rtabmap/DetectionRate': '3.0',     # Keyframe detection frequency (3Hz)
    'RGBD/LinearUpdate': '0.1',         # Update keyframe after moving 0.1m
    'RGBD/AngularUpdate': '0.05',       # Update keyframe after rotating 0.05rad, dense
frame storage during turns to maintain overlap rate

    # 【ICP anomaly filtering】
    'Icp/RangeMin': '0.2',
    'Icp/MaxTranslation': '0.15',       # Filter anomalous registration jumps larger than
0.15m per frame
    'Vis/MinInliers': '40',             # Visual feature inlier threshold
}
```